App.tsx

```
import React, {useState} from 'react';
import {
  SafeAreaView,
  View,
  Text,
  FlatList,
  StyleSheet,
  StatusBar,
  TouchableOpacity,
  TextInput,
  Dimensions,
} from 'react-native';
import menuData from './menu.json';

const {width} = Dimensions.get('window');

// Type definition for menu items
interface MenuItem {
  id: number;
  name: string;
  category: 'Veg' | 'Non-Veg' | 'Beverage';
  price: number;
  rating: number;
}

interface CartItem extends MenuItem {
  quantity: number;
}

// Get background color based on category
const getCategoryColor = (category: string): string => {
  switch (category) {
    case 'Veg':
      return '#F1F8E9';
    case 'Non-Veg':
      return '#FBE9E7';
    case 'Beverage':
      return '#E3F2FD';
    default:
      return '#FFFFFF';
  }
};

// Get veg/non-veg indicator color
const getIndicatorColor = (category: string): string => {
  switch (category) {
    case 'Veg':
      return '#2E7D32';
```

```
    case 'Non-Veg':
      return '#C62828';
    case 'Beverage':
      return '#1565C0';
    default:
      return '#666';
  }
};

// Get food emoji based on name/category
const getFoodEmoji = (name: string, category: string): string => {
  const nameLower = name.toLowerCase();
  if (nameLower.includes('biryani')) return '🍛';
  if (nameLower.includes('chicken')) return '🍗';
  if (nameLower.includes('mutton')) return '🥩';
  if (nameLower.includes('paneer')) return '🧀';
  if (nameLower.includes('dal')) return '🥘';
  if (nameLower.includes('rice')) return '🍚';
  if (nameLower.includes('coffee')) return '☕';
  if (nameLower.includes('lassi')) return '🥛';
  if (nameLower.includes('lime') || nameLower.includes('soda')) return '🍋';
  if (category === 'Beverage') return '🥤';
  if (category === 'Veg') return '🥗';
  return '🍽';
};

// Format price with rupee symbol
const formatPrice = (price: number): string => {
  return `₹${price}`;
};

// Veg/Non-Veg Indicator Component
const CategoryIndicator: React.FC<{category: string}> = ({category}) => (
  <View style={[styles.indicator, {borderColor: getIndicatorColor(category)}]}>
    <View
      style={[
        styles.indicatorDot,
        {backgroundColor: getIndicatorColor(category)},
      ]}
    />
  </View>
);

// Card component for each food item
const FoodCard: React.FC<{
  item: MenuItem;
  onAddToCart: (item: MenuItem) => void;
  cartQuantity: number;
  onUpdateQuantity: (id: number, delta: number) => void;
}> = ({item, onAddToCart, cartQuantity, onUpdateQuantity}) => {
```

```
  return (
    <View
      style={[styles.card, {backgroundColor: getCategoryColor(item.category)}]}>
      {/* Food Image Placeholder */}
      <View style={styles.imageContainer}>
        <Text style={styles.foodEmoji}>
          {getFoodEmoji(item.name, item.category)}
        </Text>
        <View style={styles.ratingBadge}>
          <Text style={styles.ratingText}>★ {item.rating}</Text>
        </View>
      </View>

      {/* Card Content */}
      <View style={styles.cardContent}>
        <View style={styles.cardHeader}>
          <CategoryIndicator category={item.category} />
          <Text style={styles.categoryLabel}>{item.category}</Text>
        </View>

        <Text style={styles.foodName} numberOfLines={2}>
          {item.name}
        </Text>

        <View style={styles.cardFooter}>
          <Text style={styles.price}>{formatPrice(item.price)}</Text>

          {cartQuantity === 0 ? (
            <TouchableOpacity
              style={styles.addButton}
              onPress={() => onAddToCart(item)}
              activeOpacity={0.7}>
              <Text style={styles.addButtonText}>ADD</Text>
            </TouchableOpacity>
          ) : (
            <View style={styles.quantityContainer}>
              <TouchableOpacity
                style={styles.quantityButton}
                onPress={() => onUpdateQuantity(item.id, -1)}>
                <Text style={styles.quantityButtonText}>−</Text>
              </TouchableOpacity>
              <Text style={styles.quantityText}>{cartQuantity}</Text>
              <TouchableOpacity
                style={styles.quantityButton}
                onPress={() => onUpdateQuantity(item.id, 1)}>
                <Text style={styles.quantityButtonText}>+</Text>
              </TouchableOpacity>
            </View>
          )}
        </View>
```

```
      </View>
    </View>
  );
};

// App Header Component
const AppHeader: React.FC<{
  searchQuery: string;
  setSearchQuery: (query: string) => void;
}> = ({searchQuery, setSearchQuery}) => (
  <View style={styles.appHeader}>
    <View style={styles.locationRow}>
      <Text style={styles.locationIcon}>📍</Text>
      <View>
        <Text style={styles.deliverTo}>DELIVER TO</Text>
        <Text style={styles.locationText}>Home - Bangalore, 560001 ▼</Text>
      </View>
    </View>
    <View style={styles.searchContainer}>
      <Text style={styles.searchIcon}>🔍</Text>
      <TextInput
        style={styles.searchInput}
        placeholder="Search for dishes..."
        placeholderTextColor="#999"
        value={searchQuery}
        onChangeText={setSearchQuery}
      />
    </View>
  </View>
);

// Category Filter Component
const CategoryFilter: React.FC<{
  selectedCategory: string;
  onSelectCategory: (category: string) => void;
}> = ({selectedCategory, onSelectCategory}) => {
  const categories = ['All', 'Veg', 'Non-Veg', 'Beverage'];

  return (
    <View style={styles.filterContainer}>
      {categories.map(category => (
        <TouchableOpacity
          key={category}
          style={[
            styles.filterButton,
            selectedCategory === category && styles.filterButtonActive,
          ]}
          onPress={() => onSelectCategory(category)}>
          <Text
            style={[
```

```
              styles.filterText,
              selectedCategory === category && styles.filterTextActive,
            ]}>
            {category}
          </Text>
        </TouchableOpacity>
      ))}
    </View>
  );
};

// List Header Component
const ListHeader: React.FC<{itemCount: number}> = ({itemCount}) => (
  <View style={styles.listHeader}>
    <Text style={styles.menuTitle}>🍴 Our Menu</Text>
    <Text style={styles.itemCount}>{itemCount} items</Text>
  </View>
);

// List Footer Component
const ListFooter = () => (
  <View style={styles.footer}>
    <View style={styles.footerDivider} />
    <Text style={styles.footerText}>Thank You for Visiting! 🙏</Text>
    <Text style={styles.footerSubtext}>
      Made with ❤️ for food lovers
    </Text>
  </View>
);

// Separator component
const ItemSeparator = () => <View style={styles.separator} />;

// Empty list component
const EmptyList = () => (
  <View style={styles.emptyContainer}>
    <Text style={styles.emptyEmoji}>🍽️</Text>
    <Text style={styles.emptyText}>No dishes found</Text>
    <Text style={styles.emptySubtext}>
      Try adjusting your search or filters
    </Text>
  </View>
);

// Cart Summary Component
const CartSummary: React.FC<{
  cart: CartItem[];
  total: number;
}> = ({cart, total}) => {
  const itemCount = cart.reduce((sum, item) => sum + item.quantity, 0);
```

```

  if (itemCount === 0) return null;

  return (
    <View style={styles.cartSummary}>
      <View style={styles.cartInfo}>
        <Text style={styles.cartItemCount}>{itemCount} item(s)</Text>
        <Text style={styles.cartTotal}>{formatPrice(total)}</Text>
      </View>
      <TouchableOpacity style={styles.checkoutButton} activeOpacity={0.8}>
        <Text style={styles.checkoutText}>View Cart</Text>
        <Text style={styles.checkoutArrow}>→</Text>
      </TouchableOpacity>
    </View>
  );
};

const App = (): React.JSX.Element => {
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [cart, setCart] = useState<CartItem[]>([]);

  // Filter menu items
  const filteredData = (menuData as MenuItem[]).filter(item => {
    const matchesSearch = item.name
      .toLowerCase()
      .includes(searchQuery.toLowerCase());
    const matchesCategory =
      selectedCategory === 'All' || item.category === selectedCategory;
    return matchesSearch && matchesCategory;
  });

  // Cart functions
  const addToCart = (item: MenuItem) => {
    setCart(prevCart => [...prevCart, {...item, quantity: 1}]);
  };

  const updateQuantity = (id: number, delta: number) => {
    setCart(prevCart => {
      return prevCart
        .map(item => {
          if (item.id === id) {
            const newQuantity = item.quantity + delta;
            return {...item, quantity: newQuantity};
          }
          return item;
        })
        .filter(item => item.quantity > 0);
    });
  };
```

```

  const getCartQuantity = (id: number): number => {
    const item = cart.find(cartItem => cartItem.id === id);
    return item ? item.quantity : 0;
  };

  const cartTotal = cart.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0,
  );

  const renderItem = ({item}: {item: MenuItem}) => (
    <FoodCard
      item={item}
      onAddToCart={addToCart}
      cartQuantity={getCartQuantity(item.id)}
      onUpdateQuantity={updateQuantity}
    />
  );

  const keyExtractor = (item: MenuItem) => item.id.toString();

  return (
    <SafeAreaView style={styles.container}>
      <StatusBar barStyle="dark-content" backgroundColor="#FFFFFF" />

      <AppHeader searchQuery={searchQuery} setSearchQuery={setSearchQuery} />
      <CategoryFilter
        selectedCategory={selectedCategory}
        onSelectCategory={setSelectedCategory}
      />

      <FlatList
        data={filteredData}
        renderItem={renderItem}
        keyExtractor={keyExtractor}
        horizontal={true}
        showsHorizontalScrollIndicator={false}
        ListHeaderComponent={<ListHeader itemCount={filteredData.length} />}
        ListFooterComponent={ListFooter}
        ItemSeparatorComponent={ItemSeparator}
        ListEmptyComponent={EmptyList}
        contentContainerStyle={styles.listContainer}
      />

      <CartSummary cart={cart} total={cartTotal} />
    </SafeAreaView>
  );
};

```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#F5F5F5',
  },
  appHeader: {
    backgroundColor: '#FFFFFF',
    paddingHorizontal: 16,
    paddingTop: 12,
    paddingBottom: 12,
    borderBottomWidth: 1,
    borderBottomColor: '#EEEEEE',
  },
  locationRow: {
    flexDirection: 'row',
    alignItems: 'center',
    marginBottom: 12,
  },
  locationIcon: {
    fontSize: 20,
    marginRight: 8,
  },
  deliverTo: {
    fontSize: 11,
    color: '#666',
    fontWeight: '600',
    letterSpacing: 0.5,
  },
  locationText: {
    fontSize: 15,
    fontWeight: '700',
    color: '#333',
  },
  searchContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: '#F5F5F5',
    borderRadius: 12,
    paddingHorizontal: 12,
    height: 44,
  },
  searchIcon: {
    fontSize: 16,
    marginRight: 8,
  },
  searchInput: {
    flex: 1,
    fontSize: 15,
    color: '#333',
    padding: 0,
```

```
  },
  filterContainer: {
    flexDirection: 'row',
    backgroundColor: '#FFFFFF',
    paddingHorizontal: 16,
    paddingVertical: 12,
    gap: 8,
  },
  filterButton: {
    paddingHorizontal: 16,
    paddingVertical: 8,
    borderRadius: 20,
    backgroundColor: '#F5F5F5',
    borderWidth: 1,
    borderColor: '#E0E0E0',
  },
  filterButtonActive: {
    backgroundColor: '#FF5722',
    borderColor: '#FF5722',
  },
  filterText: {
    fontSize: 13,
    fontWeight: '600',
    color: '#666',
  },
  filterTextActive: {
    color: '#FFFFFF',
  },
  listContainer: {
    paddingVertical: 16,
    paddingHorizontal: 8,
    alignItems: 'center',
    minHeight: 300,
  },
  listHeader: {
    paddingHorizontal: 12,
    paddingRight: 20,
  },
  menuTitle: {
    fontSize: 20,
    fontWeight: '800',
    color: '#333',
  },
  itemCount: {
    fontSize: 13,
    color: '#888',
    marginTop: 2,
  },
  footer: {
    paddingHorizontal: 20,
```

```
    paddingVertical: 24,
    alignItems: 'center',
  },
  footerDivider: {
    width: 40,
    height: 3,
    backgroundColor: '#DDD',
    borderRadius: 2,
    marginBottom: 16,
  },
  footerText: {
    fontSize: 16,
    fontWeight: '700',
    color: '#4CAF50',
  },
  footerSubtext: {
    fontSize: 12,
    color: '#999',
    marginTop: 4,
  },
  card: {
    width: width * 0.55,
    borderRadius: 16,
    backgroundColor: '#FFFFFF',
    shadowColor: '#000',
    shadowOffset: {width: 0, height: 4},
    shadowOpacity: 0.1,
    shadowRadius: 12,
    elevation: 5,
    overflow: 'hidden',
  },
  imageContainer: {
    height: 120,
    backgroundColor: 'rgba(0,0,0,0.03)',
    justifyContent: 'center',
    alignItems: 'center',
    position: 'relative',
  },
  foodEmoji: {
    fontSize: 56,
  },
  ratingBadge: {
    position: 'absolute',
    bottom: 8,
    left: 8,
    backgroundColor: '#2E7D32',
    paddingHorizontal: 8,
    paddingVertical: 4,
    borderRadius: 6,
  },
```

```
  ratingText: {
    color: '#FFFFFF',
    fontSize: 12,
    fontWeight: '700',
  },
  cardContent: {
    padding: 14,
  },
  cardHeader: {
    flexDirection: 'row',
    alignItems: 'center',
    marginBottom: 8,
  },
  indicator: {
    width: 16,
    height: 16,
    borderWidth: 2,
    borderRadius: 3,
    justifyContent: 'center',
    alignItems: 'center',
    marginRight: 6,
  },
  indicatorDot: {
    width: 8,
    height: 8,
    borderRadius: 4,
  },
  categoryLabel: {
    fontSize: 11,
    color: '#888',
    fontWeight: '600',
    textTransform: 'uppercase',
    letterSpacing: 0.5,
  },
  foodName: {
    fontSize: 16,
    fontWeight: '700',
    color: '#333',
    marginBottom: 12,
    lineHeight: 22,
  },
  cardFooter: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  price: {
    fontSize: 18,
    fontWeight: '800',
    color: '#333',
```

```
549   },
550   addButton: {
551     backgroundColor: '#FFFFFF',
552     borderWidth: 1,
553     borderColor: '#FF5722',
554     paddingHorizontal: 20,
555     paddingVertical: 8,
556     borderRadius: 8,
557   },
558   addButtonText: {
559     color: '#FF5722',
560     fontSize: 14,
561     fontWeight: '800',
562   },
563   quantityContainer: {
564     flexDirection: 'row',
565     alignItems: 'center',
566     backgroundColor: '#FF5722',
567     borderRadius: 8,
568     overflow: 'hidden',
569   },
570   quantityButton: {
571     paddingHorizontal: 12,
572     paddingVertical: 8,
573   },
574   quantityButtonText: {
575     color: '#FFFFFF',
576     fontSize: 18,
577     fontWeight: '700',
578   },
579   quantityText: {
580     color: '#FFFFFF',
581     fontSize: 14,
582     fontWeight: '700',
583     minWidth: 20,
584     textAlign: 'center',
585   },
586   separator: {
587     width: 14,
588   },
589   emptyContainer: {
590     flex: 1,
591     justifyContent: 'center',
592     alignItems: 'center',
593     paddingVertical: 60,
594     paddingHorizontal: 40,
595   },
596   emptyEmoji: {
597     fontSize: 64,
598     marginBottom: 16,
```

```
  },
  emptyText: {
    fontSize: 18,
    fontWeight: '700',
    color: '#666',
  },
  emptySubtext: {
    fontSize: 14,
    color: '#999',
    marginTop: 8,
    textAlign: 'center',
  },
  cartSummary: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    backgroundColor: '#FF5722',
    paddingHorizontal: 20,
    paddingVertical: 14,
    marginHorizontal: 16,
    marginBottom: 16,
    borderRadius: 12,
    shadowColor: '#FF5722',
    shadowOffset: {width: 0, height: 4},
    shadowOpacity: 0.3,
    shadowRadius: 8,
    elevation: 6,
  },
  cartInfo: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
  },
  cartItemCount: {
    color: '#FFFFFF',
    fontSize: 14,
    fontWeight: '600',
  },
  cartTotal: {
    color: '#FFFFFF',
    fontSize: 18,
    fontWeight: '800',
  },
  checkoutButton: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
  },
  checkoutText: {
    color: '#FFFFFF',
```

```
    fontSize: 15,
    fontWeight: '700',
  },
  checkoutArrow: {
    color: '#FFFFFF',
    fontSize: 18,
    fontWeight: '700',
  },
});

export default App;

```